

LAPORAN
WORKSHOP SISTEM INFORMASI WEB SERVER
(TIF330805)
SEMESTER III



TUGAS PRAKTIKUM ACARA 9

Nama:
RAJATA PRINGGIT BINTANG BIHAR

NIM:
E41250264

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)
TEKNIK INFORMATIKA KAB. NGANJUK
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER
TAHUN 2026

PENDAHULUAN

1. Latar Belakang

Sistem Informasi Akademik yang dikembangkan pada praktikum sebelumnya menerapkan pola MVC (Model-View-Controller). Pada bentuk awalnya, Controller dapat membuat koneksi database secara langsung menggunakan new PDO di setiap method. Cara ini membuat kode sulit diuji dan sulit diganti implementasinya, karena Controller terikat langsung pada cara koneksi database dibuat.

Praktikum Acara 9 membahas OOP lanjutan untuk mengatasi masalah tersebut, yaitu Object Composition, Dependency Injection melalui constructor, serta Getter dan Setter sebagai penerapan enkapsulasi. Dengan ketiga konsep ini, tanggung jawab setiap class menjadi lebih jelas: class Database mengelola koneksi, MahasiswaRepository mengelola akses data, dan MahasiswaController hanya mengatur alur aplikasi.

2. Tujuan

1. Membuat class Database yang mengelola koneksi database menggunakan PDO.
2. Menerapkan constructor injection pada MahasiswaRepository dan MahasiswaController.
3. Menerapkan enkapsulasi pada class Mahasiswa dengan atribut private serta getter dan setter yang memuat validasi.
4. Memastikan fitur tambah, ubah, tampil, dan hapus data tetap berjalan setelah refactoring.

3. Dasar Teori

Object Composition

Object Composition adalah teknik menyusun objek dengan cara membuat sebuah class memiliki objek dari class lain, bukan mewarisi terlalu dalam. Pada aplikasi ini Controller memiliki Repository, dan Repository memiliki objek Database yang membungkus PDO.

Dependency Injection

Dependency Injection adalah teknik menyerahkan objek yang dibutuhkan sebuah class dari luar, biasanya melalui constructor, sehingga class tersebut tidak membuat sendiri dependency-nya. Hasilnya kode lebih fleksibel karena dependency dapat diganti, dan lebih mudah diuji (testable).

Getter dan Setter

Getter dan setter adalah method untuk membaca dan mengubah atribut yang dideklarasikan private (enkapsulasi). Setter dapat menambahkan validasi sehingga data yang tidak valid ditolak sebelum disimpan di dalam objek.

Hasil Praktik

1. Implementasi

a. Class Database

Class Database (app/Core/Database.php) bertugas mengelola koneksi ke database memakai PDO dan menyediakan objek PDO melalui method getConnection(). Mode error diatur ke exception agar kesalahan query dapat ditangani dengan try-catch.

```
class Database
{
    private PDO $pdo;

    public function __construct()
    {
        $config = require __DIR__ . '/../../config/database.php';

        try {
            $this->pdo = new PDO(
                "mysql:host={$config['host']};dbname={$config['dbname']};cha
                rset={$config['charset']}",
                $config['username'],
                $config['password']
            );

            $this->pdo->setAttribute(
                PDO::ATTR_ERRMODE,
                PDO::ERRMODE_EXCEPTION
            );
        } catch (PDOException $e) {
            die('Koneksi database gagal: ' . $e->getMessage());
        }
    }

    public function getConnection(): PDO
    {
        return $this->pdo;
    }
}
```

b. Class MahasiswaRepository

MahasiswaRepository (app/Repositories/MahasiswaRepository.php) menerima objek Database melalui constructor (constructor injection) dan memakainya untuk seluruh operasi CRUD data mahasiswa.

```

class MahasiswaRepository
{
    private Database $database;

    public function __construct(Database $database)
    {
        $this->database = $database;
    }

    private function db(): PDO
    {
        return $this->database->getConnection();
    }

    public function deleteByNim(string $nim): bool
    {
        $stmt = $this->db()->prepare(
            "DELETE FROM mahasiswa WHERE nim = :nim"
        );

        return $stmt->execute([
            'nim' => $nim
        ]);
    }
}

```

c. Class MahasiswaController

MahasiswaController tidak lagi membuat koneksi database. Objek MahasiswaRepository diberikan melalui constructor, lalu dipakai untuk mengambil dan mengubah data.

```

class MahasiswaController extends Controller
{
    private MahasiswaRepository $repo;

    public function __construct(MahasiswaRepository $repo)
    {
        $this->repo = $repo;
    }

    public function index(): void
    {
        $keyword = trim($_GET['q'] ?? '');

        $this->render(
            __DIR__ . '/../Views/mahasiswa/index.php',
            [
                'daftarMahasiswa' => $this->repo->all($keyword),
            ]
        );
    }
}

```

```

        'keyword' => $keyword
    ],
    'mahasiswa'
);
}

```

d. Class Mahasiswa (getter, setter, dan validasi)

Seluruh atribut class Mahasiswa dijadikan private. Data dibaca melalui getter dan diubah melalui setter. Setter setNim() menolak NIM yang bukan angka, dan setNama() menolak nama yang kosong dengan melempar `InvalidArgumentException`.

```

class Mahasiswa
{
    private string $nim;
    private string $nama;
    private string $prodi;
    private string $angkatan;
    private string $status;
    public function getNim(): string
    {
        return $this->nim;
    }

    public function setNim(string $nim): void
    {
        if (!preg_match('/^\d+$/ ', $nim)) {
            throw new \InvalidArgumentException('NIM harus berupa angka. ');
        }

        $this->nim = $nim;
    }

    public function getNama(): string
    {
        return $this->nama;
    }

    public function setNama(string $nama): void
    {
        $nama = trim($nama);

        if ($nama === ' ') {
            throw new \InvalidArgumentException('Nama mahasiswa tidak boleh
kosong. ');
        }

        $this->nama = $nama;
    }
}

```

```

    }

    public function getProdi(): string
    {
        return $this->prodi;
    }

    public function setProdi(string $prodi): void
    {
        $prodi = trim($prodi);

        if ($prodi === '') {
            throw new \InvalidArgumentException('Prodi tidak boleh
kosong. ');
        }

        $this->prodi = $prodi;
    }

    public function getAngkatan(): string
    {
        return $this->angkatan;
    }

    public function setAngkatan(string $angkatan): void
    {
        $angkatan = trim($angkatan);

        if (!preg_match('/^\d{4}$/', $angkatan)) {
            throw new \InvalidArgumentException('Angkatan harus terdiri dari
4 angka. ');
        }

        $this->angkatan = $angkatan;
    }
}

```

e. Perakitan dependency

Objek Database dan MahasiswaRepository dibuat pada Router::makeController() lalu disuntikkan ke MahasiswaController. Dengan demikian, controller tidak pernah membuat koneksi database sendiri.

```

private function makeController(string $controllerClass): object
{
    if ($controllerClass === 'App\\Controllers\\MahasiswaController') {
        $database = new Database();
        $repository = new MahasiswaRepository($database);
    }
}

```

```

        return new $controllerClass($repository);
    }

    return new $controllerClass();
}

```

2. Pemetaan dengan Tugas Praktikum

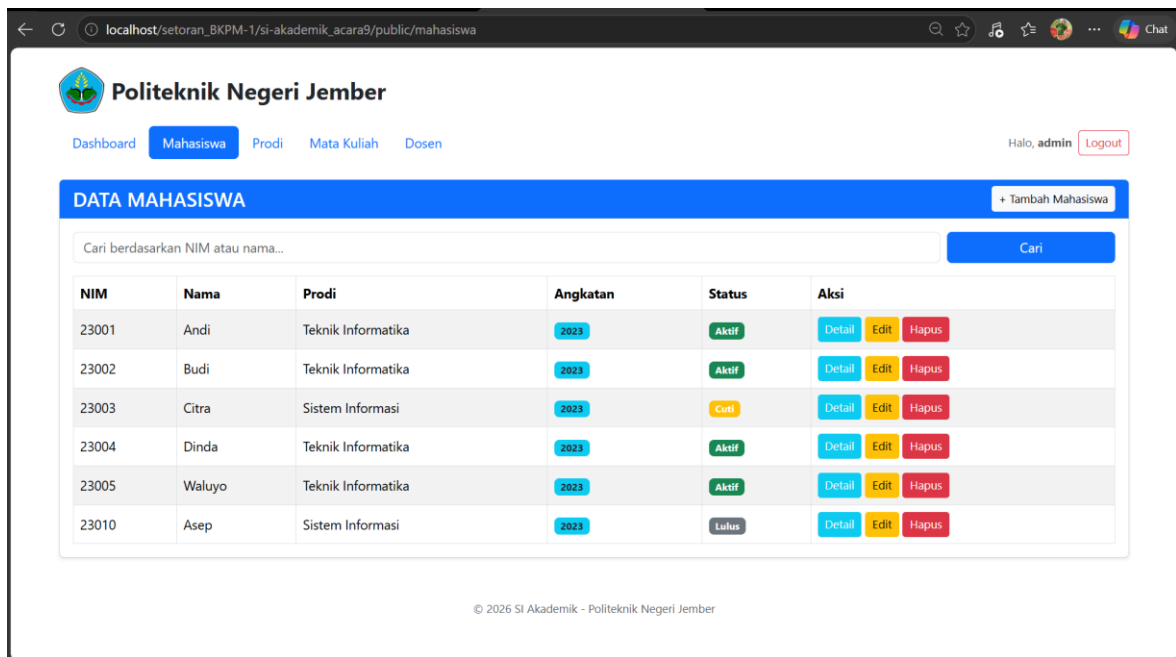
Tabel 1. Pemetaan tugas pada modul dengan implementasi

No	Tugas pada modul	Implementasi	File
1	Membuat class Database untuk mengelola koneksi PDO	Class Database dengan getConnection() dan mode ERRMODE_EXCEPTION	app/Core/Database.php
2	MahasiswaRepository menerima Database melalui constructor dan melakukan CRUD	Constructor injection; method all, findByNim, create, updateByNim, deleteByNim	app/Repositories/MahasiswaRepository.php
3	MahasiswaController menerima MahasiswaRepository melalui constructor	Constructor injection; controller tidak membuat koneksi langsung	app/Controllers/MahasiswaController.php
4	Atribut Mahasiswa private, dengan getter dan setter	Atribut nim, nama, prodi, angkatan, dan status private	app/Models/Mahasiswa.php
5	Validasi sederhana pada setter	NIM harus angka, nama tidak boleh kosong; ditambah validasi prodi, angkatan, dan status	app/Models/Mahasiswa.php
6	CRUD tetap berjalan setelah refactoring	Diuji pada browser (lihat bagian 3)	-

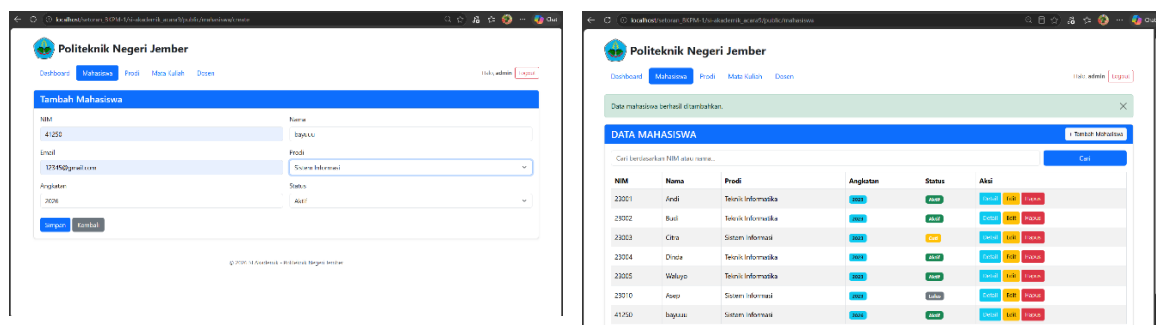
3. Pengujian Aplikasi

a. Pengujian fitur CRUD

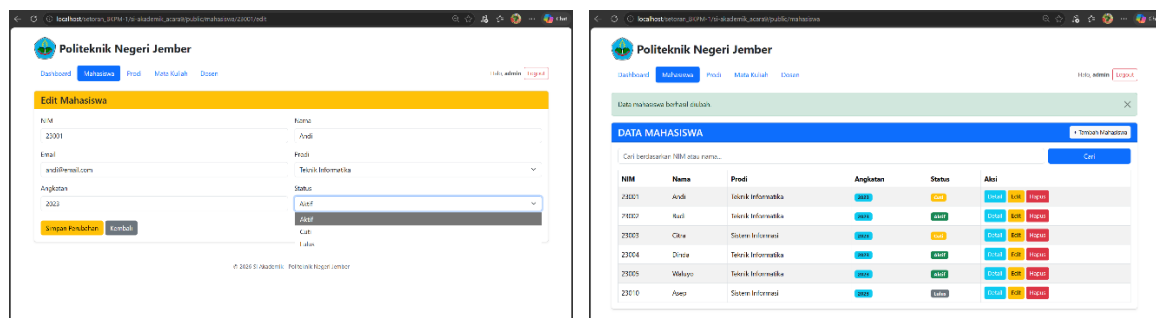
Aplikasi dijalankan pada XAMPP, kemudian keempat fitur diuji satu per satu setelah login. Seluruh fitur berjalan dengan baik setelah refactoring.



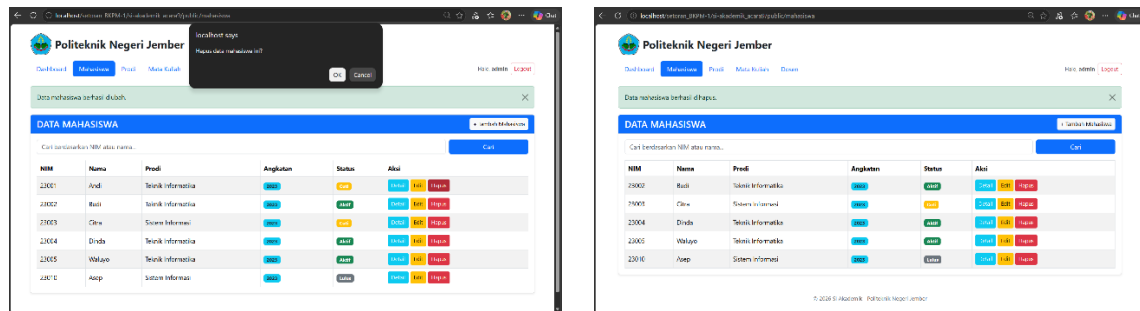
Gambar 1. Tampil data mahasiswa



Gambar 2. Tambah data mahasiswa



Gambar 3. Ubah data mahasiswa



Gambar 4. Hapus data mahasiswa

b. Pengujian validasi pada setter

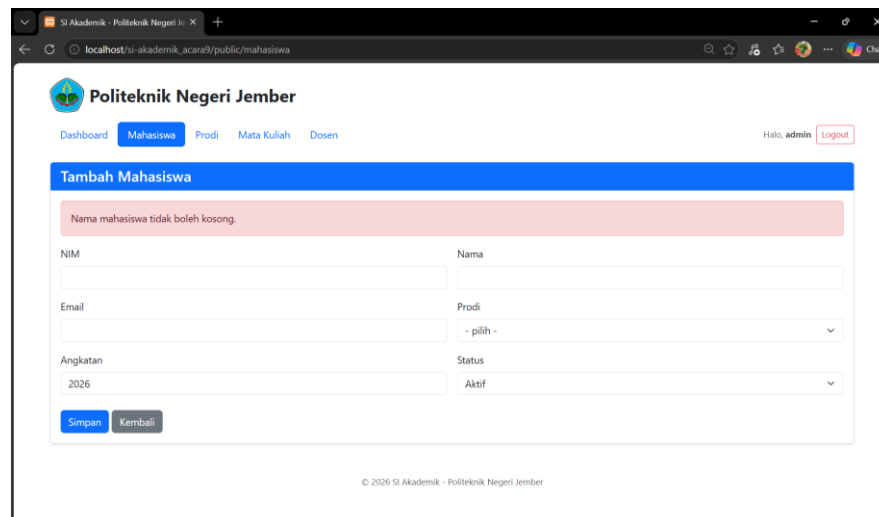
Validasi diuji dengan memasukkan data yang tidak sesuai pada form Tambah Mahasiswa. Setter melempar exception, controller menangkapnya, lalu pesan kesalahan ditampilkan dan data tidak tersimpan.

The screenshot shows the 'Tambah Mahasiswa' form with a red error message at the top: 'NIM harus berupa angka.' The form fields are: NIM (text input), Nama (text input), Email (text input), Prodi (dropdown menu), Angkatan (text input with value '2026'), and Status (dropdown menu with value 'Aktif'). There are 'Simpan' and 'Kembali' buttons at the bottom. The footer reads '© 2026 SI Akademik - Politeknik Negeri Jember'.

Gambar 5. NIM berisi huruf ditolak oleh setNim()

The screenshot shows the 'Tambah Mahasiswa' form with a red error message at the top: 'Angkatan harus terdiri dari 4 angka.' The form fields are: NIM (text input), Nama (text input), Email (text input), Prodi (dropdown menu), Angkatan (text input with value '2026'), and Status (dropdown menu with value 'Aktif'). There are 'Simpan' and 'Kembali' buttons at the bottom. The footer reads '© 2026 SI Akademik - Politeknik Negeri Jember'.

Gambar 6. Angkatan yang bukan 4 angka ditolak oleh setAngkatan()



Gambar 7. Nama kosong ditolak oleh `setNama()`

4. Pembahasan

Sebelum refactoring, koneksi database dibuat di dalam Controller sehingga Controller mengetahui detail cara menyambung ke database. Setelah Dependency Injection diterapkan, MahasiswaController hanya menerima objek MahasiswaRepository melalui constructor dan memanggil method seperti `all()` tanpa mengetahui bagaimana query dijalankan. Jika suatu saat implementasi repository diganti, misalnya untuk keperluan pengujian, Controller tidak perlu diubah.

Object Composition terlihat pada rantai kepemilikan Controller, Repository, Database, dan PDO. Setiap class memiliki satu tanggung jawab sehingga perubahan pada satu lapisan berdampak kecil pada lapisan lain.

Penerapan getter dan setter menjaga data tetap valid. Atribut Mahasiswa tidak dapat diubah langsung dari luar class; perubahan harus melalui setter yang melempar `InvalidArgumentException` bila data tidak sesuai. Controller menangkap exception tersebut lalu menampilkan pesan kesalahan pada form, sehingga data tidak valid tidak pernah sampai ke database. Pengujian pada bagian 3b membuktikan bahwa validasi NIM, nama, dan angkatan berjalan.

Sebagai catatan, perakitan objek saat ini masih dilakukan di Router. Untuk pengembangan berikutnya, perakitan tersebut dapat dipindahkan ke sebuah container atau factory tersendiri agar Router hanya bertugas mencocokkan rute.

KESIMPULAN

1. Class Database, MahasiswaRepository, dan MahasiswaController berhasil disusun dengan constructor injection sehingga Controller tidak lagi membuat koneksi database secara langsung.
2. Object Composition membuat tanggung jawab setiap class terpisah, yaitu Controller mengatur alur, Repository mengelola data, dan Database mengelola koneksi.
3. Getter dan setter pada class Mahasiswa memastikan data yang masuk valid melalui validasi NIM, nama, prodi, angkatan, dan status.
4. Fitur tampil, tambah, ubah, dan hapus data tetap berjalan dengan baik setelah refactoring.

DAFTAR PUSTAKA

Pratama, R. A., & Hasanah, Q. (2026). Buku Kerja Praktik Mahasiswa (BKPM) Workshop Sistem Informasi Web Server (TIF330805). Program Studi D4 Teknik Informatika PSDKU Nganjuk, Jurusan Teknologi Informasi, Politeknik Negeri Jember.